

Name bhavya rana

Sapid 590034689

Batch b -15

Algorithm

Main Program

1. Start.
2. Declare **choice** as an integer.
3. Display the menu:
 1. Solid square pattern
 2. Solid rectangular pattern
 3. Row-number pattern
 4. Column-number pattern
 5. Exit
4. Read **choice**.
5. Use a **switch** statement:
 - If **choice** = 1, call **solidSquare()**.
 - If **choice** = 2, call **solidRectangle()**.
 - If **choice** = 3, call **rowNumber()**.
 - If **choice** = 4, call **columnNumber()**.
 - If **choice** = 5, display an exit message.
 - Otherwise, display "**Invalid choice.**"
6. Repeat the menu until the user selects 5.
7. Stop.

1. Solid Square Pattern

1. Start **solidSquare()**.
2. Input **n**.
3. If **n** <= 0, display an error message and return.
4. Repeat **i** from 1 to **n**.
5. For each row, repeat **j** from 1 to **n**.
6. Print * .
7. After each row, print a newline.
8. Return to the main program.

2. Solid Rectangular Pattern

1. Start **solidRectangle()**.
2. Input rows **m** and columns **n**.
3. If **m** <= 0 or **n** <= 0, display an error message and return.

4. Repeat i from 1 to m .
5. For each row, repeat j from 1 to n .
6. Print * .
7. Print a newline after each row.
8. Return to the main program.

3. Row-Number Pattern

1. Start `rowNumber ( )`.
2. Input rows m and columns n .
3. Validate that both are positive.
4. Repeat i from 1 to m .
5. Repeat j from 1 to n .
6. Print the value of i .
7. Print a newline after each row.
8. Return to the main program.

4. Column-Number Pattern

1. Start `columnNumber ( )`.
2. Input rows m and columns n .
3. Validate that both are positive.
4. Repeat i from 1 to m .
5. Repeat j from 1 to n .
6. Print the value of j .
7. Print a newline after each row.
8. Return to the main program.

Test-Case Table

Test Case	Menu Choice	Input(s)	Expected Output	Actual Output	Result
1	1	$n = 4$	4×4 solid square	4×4 solid square	Pass
2	1	$n = 1$	One *	One *	Pass
3	1	$n = 0$	Invalid input message	Invalid input message	Pass
4	2	$m = 3, n =$	3×5 solid rectangle	3×5 solid	Pass
5	2	$m = -2, n = 4$	Invalid input message	Invalid input message	Pass
6	3	$m = 4, n = 4$	Rows containing 1, 2, 3, 4 respectively	As expected	Pass
7	3	$m = 2, n =$	1 1 1 and 2 2 2	As expected	Pass
8	4	$m = 4, n =$	Each row: 1 2 3 4	As expected	Pass
9	4	$m = 2, n =$	Each row: 1 2 3 4 5	As expected	Pass
10	6	—	Invalid menu choice	Invalid menu	Pass
11	5	—	Program terminates	Program	Pass

C Program

```
#include <stdio.h>

void solidSquare(void);
void solidRectangle(void);
void rowNumber(void);
void columnNumber(void);

int main(void)
{
    int choice;

    do
    {
        printf("\n----- PATTERN MENU ----- \n");
        printf("1. Solid square pattern\n");
        printf("2. Solid rectangular pattern\n");
        printf("3. Row-number pattern\n");
        printf("4. Column-number pattern\n");
        printf("5. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                solidSquare();
                break;

            case 2:
                solidRectangle();
                break;

            case 3:
                rowNumber();
                break;

            case 4:
                columnNumber();
                break;

            case 5:
```

```

        printf("Exiting the program.\n");
        break;

        default:
            printf("Invalid choice. Please enter a choice
from 1 to 5.\n");
    }

    } while (choice != 5);

    return 0;
}

/* Function to print a solid square */
void solidSquare(void)
{
    int n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Invalid input. Number of rows must be
positive.\n");
        return;
    }

    printf("\nSolid Square Pattern:\n");

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= n; j++)
        {
            printf("* ");
        }
        printf("\n");
    }
}

/* Function to print a solid rectangle */
void solidRectangle(void)
{
    int m, n, i, j;

```

```

printf("Enter the number of rows: ");
scanf("%d", &m);

printf("Enter the number of columns: ");
scanf("%d", &n);

if (m <= 0 || n <= 0)
{
    printf("Invalid input. Rows and columns must be
positive.\n");
    return;
}

printf("\nSolid Rectangular Pattern:\n");

for (i = 1; i <= m; i++)
{
    for (j = 1; j <= n; j++)
    {
        printf("* ");
    }
    printf("\n");
}

/* Function to print row-number pattern */
void rowNumber(void)
{
    int m, n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &m);

    printf("Enter the number of columns: ");
    scanf("%d", &n);

    if (m <= 0 || n <= 0)
    {
        printf("Invalid input. Rows and columns must be
positive.\n");
        return;
    }

    printf("\nRow-Number Pattern:\n");

```

```

    for (i = 1; i <= m; i++)
    {
        for (j = 1; j <= n; j++)
        {
            printf("%d ", i);
        }
        printf("\n");
    }
}

/* Function to print column-number pattern */
void columnNumber(void)
{
    int m, n, i, j;

    printf("Enter the number of rows: ");
    scanf("%d", &m);

    printf("Enter the number of columns: ");
    scanf("%d", &n);

    if (m <= 0 || n <= 0)
    {
        printf("Invalid input. Rows and columns must be positive.\n");
        return;
    }

    printf("\nColumn-Number Pattern:\n");

    for (i = 1; i <= m; i++)
    {
        for (j = 1; j <= n; j++)
        {
            printf("%d ", j);
        }
        printf("\n");
    }
}

```

Sample Execution

Choice 1 — Solid Square

```
Last login: Tue Sep 15 14:40:19 on ttys000
[bhavya@Bhavyas-MacBook-Air ~ % nano day23.cpp
[bhavya@Bhavyas-MacBook-Air ~ % clang day23.cpp -o day23
[bhavya@Bhavyas-MacBook-Air ~ % ./day23
```

```
----- PATTERN MENU -----
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 1
Enter the number of rows: 4
```

Solid Square Pattern:

```
* * * *
* * * *
* * * *
* * * *
```

```
----- PATTERN MENU -----
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
```

Choice 2 - solid rectangle

```
Enter your choice: 2
Enter the number of rows: 3
Enter the number of columns: 5
```

Solid Rectangular Pattern:

```
* * * * *
* * * * *
* * * * *
```

```
----- PATTERN MENU -----
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
```

Choice 3 - row number pattern

```
Enter your choice: 3
Enter the number of rows: 4
Enter the number of columns: 4
```

Row-Number Pattern:

```
1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4
```

----- PATTERN MENU -----

1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit

Choice 4 - column number

```
Enter your choice: 4
Enter the number of rows: 4
Enter the number of columns: 4
```

Column-Number Pattern:

```
1 2 3 4
1 2 3 4
1 2 3 4
1 2 3 4
```

----- PATTERN MENU -----

1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit

```
Enter your choice: 7
Invalid choice. Please enter a choice from 1 to 5.

----- PATTERN MENU -----
1. Solid square pattern
2. Solid rectangular pattern
3. Row-number pattern
4. Column-number pattern
5. Exit
Enter your choice: 5
Exiting the program.
bhavya@Bhavyas-MacBook-Air ~ %
```

INVALID CHOICE

this implementation satisfies the specified requirements: **switch** statement, nested loops, no arrays, parameterless functions, and **void** return type.